Fundamentals of Machine Learning

Programming Assignment Report

Ramalingham Seyon | MS26912516

Kekulawala Vidanalage Dhammika Kekulawala | MS26902784

Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)
MSc in Artificial Intelligence – 2026

Section 1 – Introduction and Problem Statement

The cardiovascular disease is among the most common causes of death in the world, with the World Health Organization (WHO) estimating 17.9 million deaths annually. Early and precise diagnosis of people with high risk of heart disease is paramount in making available clinical intervention and preventive measures in time.

The project uses supervised machine learning to predict patient risk of heart disease, given a combination of clinical, demographic, and lifestyle characteristics. Two classification algorithms are implemented and compared: Logistic Regression and Random Forest. The data is the Heart Disease Prediction Dataset is publicly available and hosted on Kaggle.

It is not just a matter of constructing correct predictive models, but also a matter of comparing the behaviour of the two algorithms on the same dataset, interpreting their predictions, and critically assessing their appropriateness to a healthcare prediction task. The project is organized in the form of a pipeline: common preprocessing, autonomous model training notebooks and a deployed Streamlit web application.

Section 2 – Table of Contents

Section 1 – Introduction and Problem Statement
Section 2 – Table of Contents
Section 3 – Dataset Description
- 3.1 Dataset Context
- 3.2 Dataset Size and Scope
- 3.3 Attribute Description
- 3.4 Why This Dataset

Section 4 – Data Preprocessing
- 4.1 Data Cleaning and Missing Value Handling
- 4.2 Feature Encoding
- 4.3 Feature Scaling
- 4.4 Dimensionality Analysis
- 4.5 Train / Test Split

Section 5 – Algorithm: Random Forest
- 5.1 Background: Decision Trees
- 5.2 How Random Forest Works
- 5.3 Why Random Forest Was Chosen
- 5.4 Implementation
- 5.5 Results

Section 6 – Algorithm: Logistic Regression
- 6.1 Background
- 6.2 How Logistic Regression Works
- 6.3 Why Logistic Regression Was Chosen
- 6.4 Implementation
- 6.5 Results

Section 7 – Comparison of Algorithms
Section 8 – Critical Analysis and Discussion
8.1 Strengths of Each Algorithm
8.2 Limitations and Key Findings
8.3 How Performance Can Be Improved
8.4 Future Work
Section 9 – Appendix (Source Code)

Section 3 – Dataset Description

3.1 Dataset Context

The dataset used in this project is the Heart Disease Prediction Dataset, sourced from Kaggle. It is accessible at: https://www.kaggle.com/datasets/oktayrdeki/heart-disease

The dataset is a records for heart disease prediction research and machine learning experimentation. It represents a wide spectrum of cardiovascular risk factors including laboratory outcomes, lifestyle, and related clinical conditions.

3.2 Dataset Size and Scope

The dataset contains 10,000 patient records and 21 columns (20 input features and 1 target label).

Parameter	Value
Total Records	10,000
Total Features	20
Target Variable	Heart Disease Status (Yes / No)
Train Set Size	8,000 records (80%)
Test Set Size	2,000 records (20%)
Class Distribution	~80% No Disease | ~20% Heart Disease

Table 1: Dataset Size and Scope

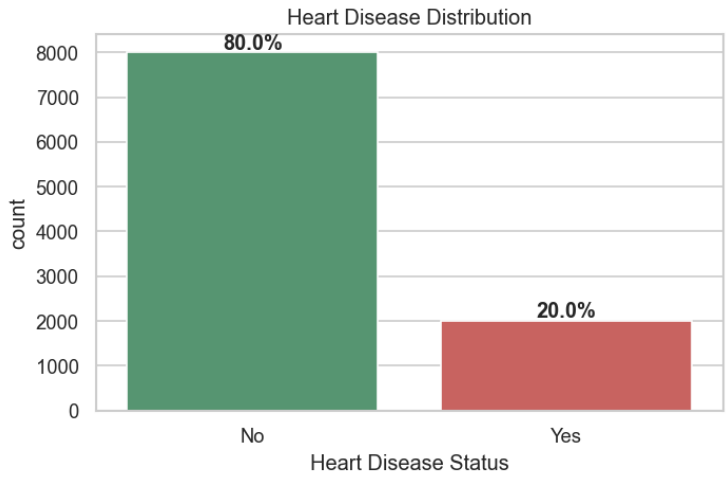

Figure 1: Heart Disease Status distribution (80% No, 20% Yes)

3.3 Attribute Description

No.	Feature	Type	Description
1	Age	Numeric	Patient age in years
2	Gender	Binary	Male or Female
3	Blood Pressure	Numeric	Systolic blood pressure (mmHg)
4	Cholesterol Level	Numeric	Total cholesterol (mg/dL)
5	Exercise Habits	Ordinal	Physical activity level (Low/Medium/High)
6	Smoking	Binary	Whether the patient smokes (Yes/No)
7	Family Heart Disease	Binary	Family history of heart disease (Yes/No)
8	Diabetes	Binary	Diabetic status (Yes/No)
9	BMI	Numeric	Body Mass Index
10	High Blood Pressure	Binary	Clinically diagnosed hypertension (Yes/No)
11	Low HDL Cholesterol	Binary	Low good cholesterol level (Yes/No)
12	High LDL Cholesterol	Binary	High bad cholesterol level (Yes/No)
13	Alcohol Consumption	Ordinal	Alcohol usage (None/Low/Medium/High)
14	Stress Level	Ordinal	Self-reported stress (Low/Medium/High)
15	Sleep Hours	Numeric	Average sleep hours per day
16	Sugar Consumption	Ordinal	Sugar intake level (Low/Medium/High)
17	Triglyceride Level	Numeric	Triglycerides (mg/dL)
18	Fasting Blood Sugar	Numeric	Fasting glucose level (mg/dL)
19	CRP Level	Numeric	C-reactive protein (mg/L) — inflammation marker
20	Homocysteine Level	Numeric	Homocysteine (umol/L) — cardiovascular risk marker

Table 2: Attribute Description of Dataset

3.4 Why This Dataset

- Medically Relevant Features: The 20 features cover well-established cardiovascular risk factors including lipid levels, blood pressure, inflammation markers (CRP, Homocysteine), and lifestyle indicators.
- Mixed Feature Types: Includes numeric, binary, and ordinal categorical variables, providing a comprehensive preprocessing challenge.
- Sufficient Size: With 10,000 records the dataset is large enough to train and evaluate ML models reliably and perform meaningful hyperparameter tuning with cross-validation.
- Publicly Available: Hosted on Kaggle, a widely recognised and credible platform for machine learning datasets.

Section 4 – Data Preprocessing

4.1 Data Cleaning and Missing Value Handling

- Missing Values: Per-column null check via isnull().sum(). A SimpleImputer was fit on the training set only: numeric columns imputed with the column median; binary and ordinal categorical columns imputed with the column mode. The fitted imputer was then applied to both train and test sets to prevent data leakage.
- Duplicate Records: Check performed — no significant duplicates found.
- Data Type Validation: Numeric columns verified as float/int; categorical columns as string. Target column confirmed to contain only Yes and No.
- Target Encoding: Heart Disease Status mapped to binary integers — No → 0, Yes → 1.
- Assertion Checks: assert statements verify zero missing values after imputation and that all categorical mappings resolved successfully (no silent NaN from unmapped values).

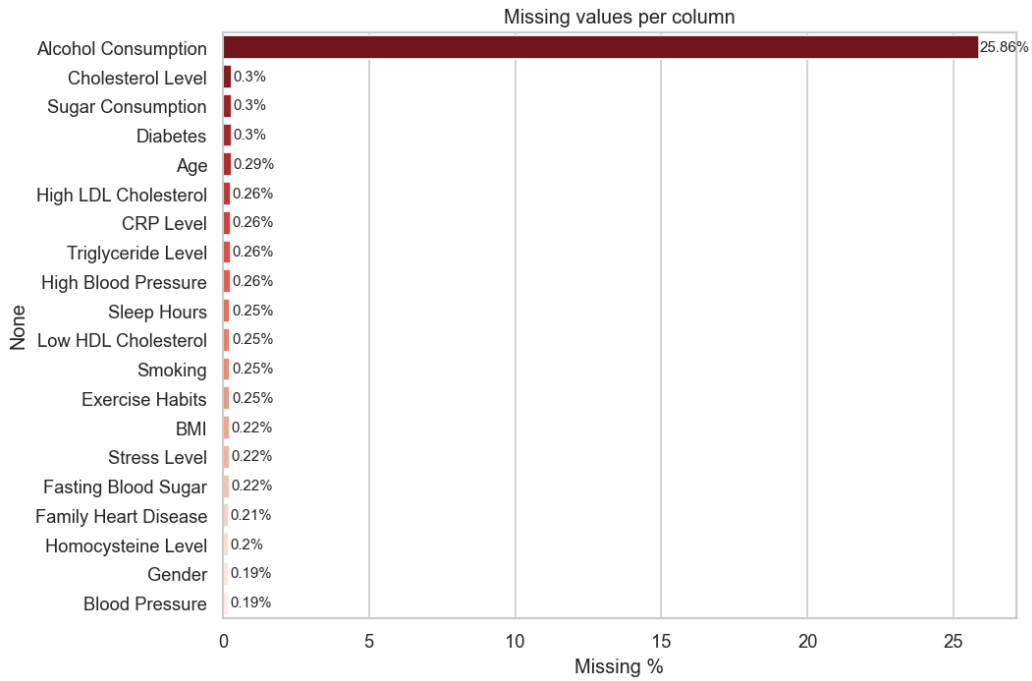

Figure 2: Missing values per column (%)

4.2 Feature Encoding

- Binary Encoding (Yes/No → 1/0, Male/Female → 1/0): Applied to Gender, Smoking, Family Heart Disease, Diabetes, High Blood Pressure, Low HDL Cholesterol, High LDL Cholesterol.
- Ordinal Encoding (Low/Medium/High → 0/1/2): Applied to Exercise Habits, Stress Level, Sugar Consumption. Alcohol Consumption encoded as None/Low/Medium/High → 0/1/2/3.

This encoding preserves the natural ordering in ordinal features, which is more appropriate than one-hot encoding for Low < Medium < High relationships.

4.3 Feature Scaling

Logistic Regression is sensitive to feature scale because it optimises a linear decision boundary. StandardScaler was applied to normalise all features to zero mean and unit variance (z-score normalisation): z = (x − mean) / std_dev. The scaler was fit exclusively on the training set and applied to both sets, preventing data leakage. The fitted scaler was serialised as scaler.pkl so that the Streamlit application can transform new patient input consistently.

4.4 Dimensionality Analysis

With 20 features the dataset is not high-dimensional. A Pearson correlation analysis confirmed no feature pair exceeded an absolute correlation coefficient of 0.5, indicating no multicollinearity. Dimensionality reduction (e.g., PCA) was therefore not applied. Correlation of each feature with the target was also computed; all values were near zero (maximum r = 0.0197 for BMI), which is discussed further in Section 8.

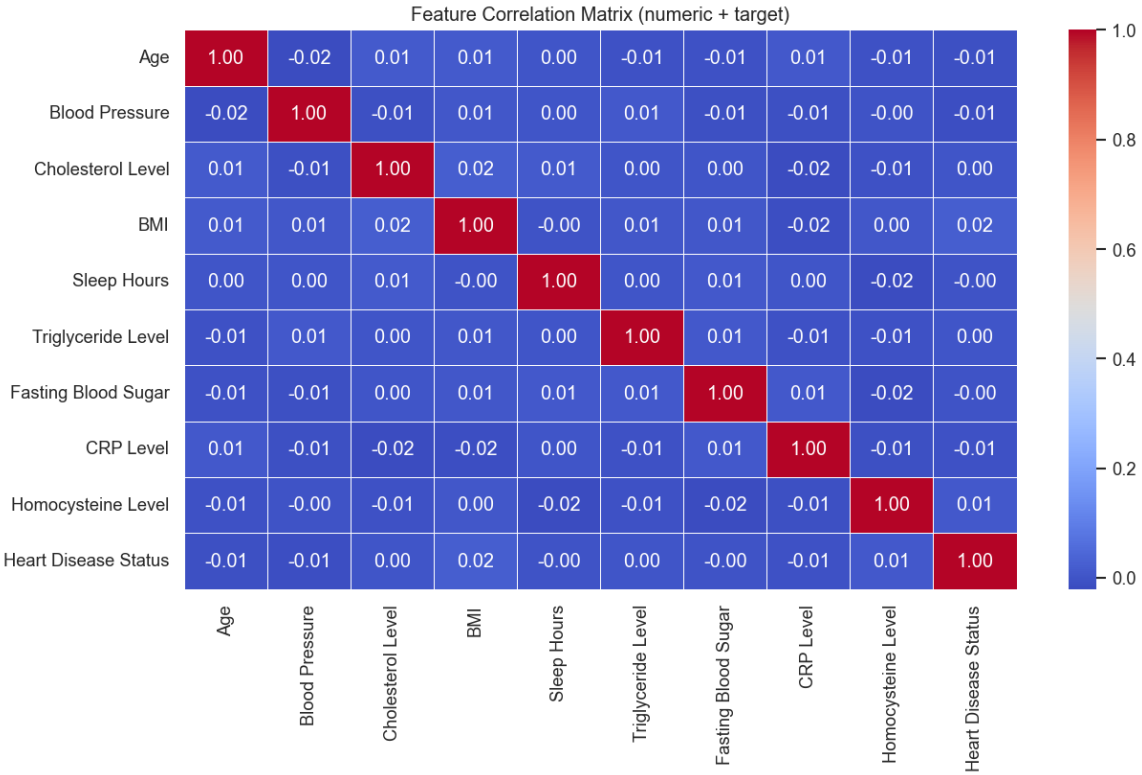

Figure 3: Pearson correlation matrix (all features + target)

4.5 Train / Test Split

The dataset was split 80/20 using train_test_split with stratify=y and random_state=42. The stratify parameter ensures both the training and test sets maintain the same class proportion as the full dataset (~80% No Disease, ~20% Heart Disease), which is critical for reliable evaluation on an imbalanced target.

Split	Records	Heart Disease Rate
Training Set	8,000	~20%
Test Set	2,000	~20%

Table 3: Train / Test Split

The split is performed once in preprocessing.ipynb and saved as CSV files. Both model notebooks load the exact same train/test data, guaranteeing a fair head-to-head comparison.

Section 5 – Algorithm: Random Forest

5.1 Background: Decision Trees

Random Forest is an ensemble learning algorithm built on decision trees. A decision tree makes predictions by recursively partitioning the feature space — asking binary questions about input features at each internal node until leaf nodes output a class prediction. While intuitive and interpretable, a single decision tree is prone to overfitting: small changes in the training data can produce substantially different trees (high variance).

5.2 How Random Forest Works

Random Forest addresses overfitting by training many diverse trees and aggregating their predictions via majority voting. Two independent sources of randomness ensure diversity:

- Bootstrap Sampling (Bagging): Each tree is trained on a different random sample of the training data, drawn with replacement. On average, each sample omits ~37% of rows.
- Feature Subsampling: At each split, only a random subset of features is considered (sqrt or log2 of the total feature count). This forces different trees to specialise on different subsets of predictors.

Combining many diverse trees reduces variance without meaningfully increasing bias — this is the principle of ensemble learning. The final prediction is the class receiving the most votes across all trees.

5.3 Why Random Forest Was Chosen

- Handles Non-linear Relationships: Cardiovascular risk is complex and non-linear. Random Forest captures these interactions naturally without explicit feature engineering.
- Robust to Outliers: Tree-based split thresholds are not distorted by extreme clinical values.
- Feature Importance: Provides two interpretable importance metrics — Gini importance (built-in) and permutation importance (model-agnostic) — valuable in healthcare for understanding which risk factors drive predictions.
- No Feature Scaling Required: Unlike distance-based or gradient-based algorithms, trees are invariant to feature scale.
- Class Imbalance Handling: The class_weight hyperparameter was included in the search grid, allowing the GridSearchCV to determine whether balanced weighting improves discrimination on this imbalanced dataset.

5.4 Implementation

5.4.1 Development Environment

Python 3.x, Jupyter Notebook. Libraries: pandas, numpy, scikit-learn, imbalanced-learn, joblib, matplotlib, seaborn.

5.4.2 Implementation Steps

- Notebook 1 (preprocessing.ipynb — Shared): Data loading, cleaning, encoding, scaling, stratified train/test split. Outputs saved as CSV files and fitted transformers saved as .pkl files, shared by both model notebooks.
- Notebook 3 (random_forest.ipynb — Dhammika): Loads pre-split data. Validates data with assert statements. Trains a baseline Random Forest. Applies full GridSearchCV with 6-parameter grid scored on ROC-AUC. Evaluates best model (confusion matrix, ROC curve, Precision-Recall curve, learning curve). Computes both Gini and permutation feature importance. Saves model, metrics, and all plots.

5.4.3 Hyperparameter Tuning

Hyperparameter	Values Tested	Description
n_estimators	[200, 400]	Number of decision trees in the forest
max_depth	[None, 10, 20]	Maximum depth of each tree (None = fully grown)
min_samples_split	[2, 5]	Minimum samples required to split an internal node
min_samples_leaf	[1, 2]	Minimum samples required in a leaf node
max_features	["sqrt", "log2"]	Features considered at each split
class_weight	[None, "balanced"]	Sample weighting strategy for class imbalance

Table 4: Random Forest Hyperparameter Grid

GridSearchCV tested all combinations using 5-fold Stratified Cross-Validation, selecting the best parameters by mean validation ROC-AUC (not accuracy). Best configuration: n_estimators=200, max_depth=20, min_samples_split=5, min_samples_leaf=2, max_features="sqrt", class_weight="balanced".

5.4.4 Cross-Validation

5-fold Stratified Cross-Validation was used, maintaining class proportions within each fold. The mean ROC-AUC across 5 folds was 0.5222 (std = 0.0196), confirming that the best model's performance is consistent across all folds and not due to a favourable random split.

5.5 Results

5.5.1 Evaluation Metrics

Accuracy, Precision, Recall, F1 Score, and ROC-AUC were computed on the held-out test set. For an imbalanced clinical dataset, ROC-AUC is the primary metric: a score of 0.5 indicates random chance; 1.0 is perfect discrimination. Accuracy alone is misleading when one class constitutes ~80% of the data.

5.5.2 Baseline Random Forest

Metric	Baseline RF (default, 100 trees)
Accuracy	0.8000
Precision	0.0000
Recall	0.0000
F1 Score	0.0000
ROC-AUC	0.5090

Table 5: Random Forest Baseline Test Results

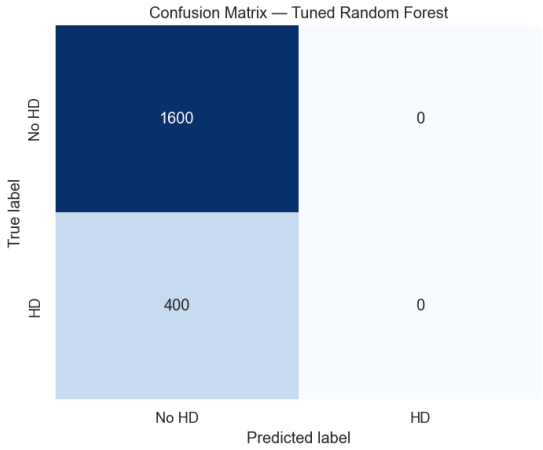

Figure 4: RF confusion matrix (TN=1600, FP=0, FN=400, TP=0)

5.5.3 Tuned Random Forest

Metric	Tuned RF (GridSearchCV best)
Accuracy	0.8000
Precision	0.0000
Recall	0.0000
F1 Score	0.0000
ROC-AUC	0.4905

Table 6: Random Forest Tuned Test Results

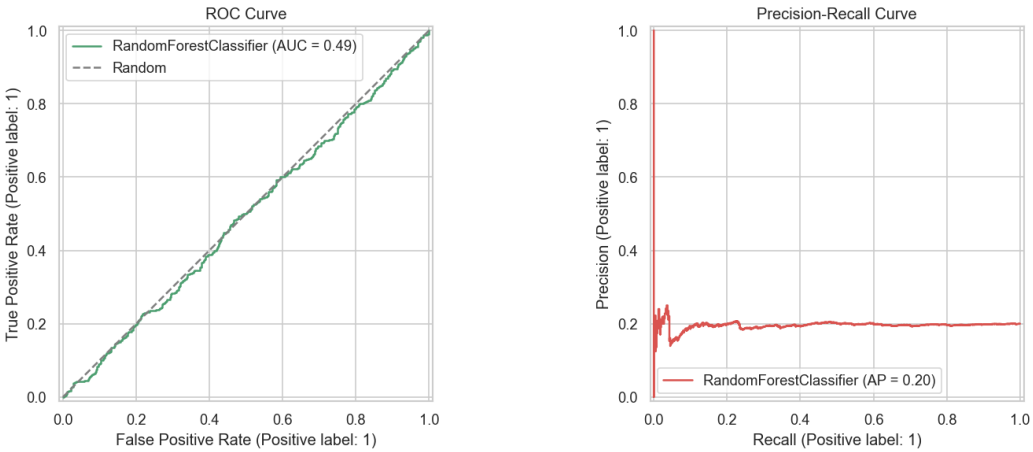

Figure 5: RF ROC curve (AUC=0.49) and Precision-Recall curve

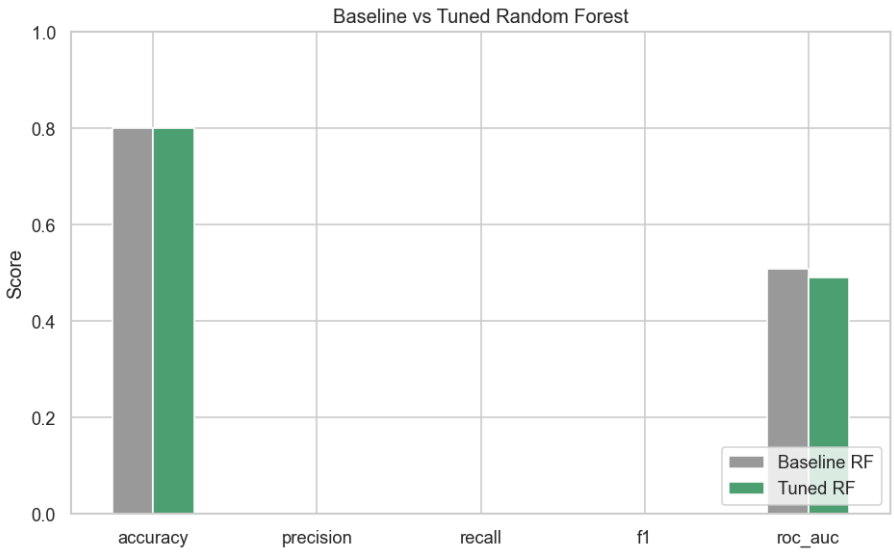

Figure 6: Baseline vs Tuned RF metrics comparison

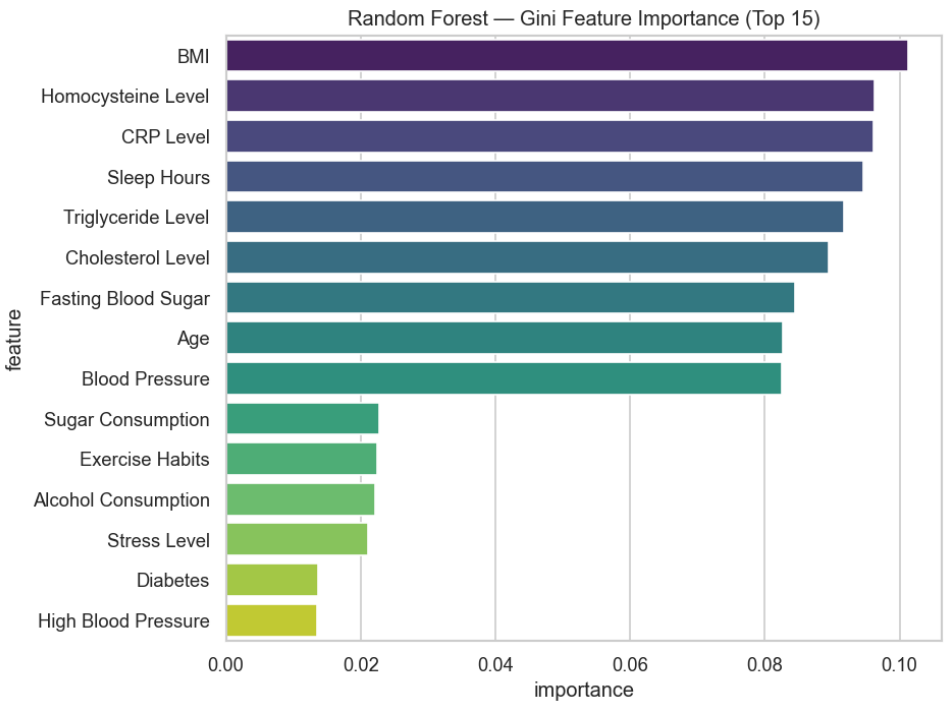

Figure 7: RF Gini feature importance (Top 15)

	Predicted: No Disease	**Predicted: Heart Disease**
Actual: No Disease	TN = 1600	FP = 0
Actual: Heart Disease	FN = 400	TP = 0

Table 7: Random Forest Confusion Matrix (Tuned Model)

Interpretation: Both the baseline and tuned Random Forest predicted every patient as No Disease (Recall = 0.000). The 80.00% accuracy simply mirrors the majority class proportion. A ROC-AUC of 0.4905 confirms the model performs at random chance — it found no discriminative signal. Top Gini feature importances: BMI (10.1%), Homocysteine Level (9.6%),

CRP Level (9.6%), Sleep Hours (9.5%), Triglyceride Level (9.2%). The near-uniform distribution of importance scores further suggests no single feature is informative.

Section 6 – Algorithm: Logistic Regression

6.1 Background

Logistic Regression is one of the foundational supervised machine learning algorithms for binary classification. Despite its name it is a classification — not a regression — algorithm. It models the probability that a given observation belongs to the positive class using a sigmoid transformation of a linear predictor.

6.2 How Logistic Regression Works

Logistic Regression models the log-odds of the positive class as a linear combination of input features:

```
log(p / (1 - p))  =  B0 + B1*x1 + B2*x2 + ... + Bn*xn
```

The log-odds (logit) is converted to a probability via the sigmoid function:

```
p  =  1 / (1 + e^(-z))
```

A threshold (default 0.5) converts the probability to a binary prediction. Regularisation penalises large coefficients to reduce overfitting. This project tested both L1 (Lasso) and L2 (Ridge) regularisation:

L2 (Ridge): penalises the sum of squared coefficients — shrinks all coefficients toward zero but rarely sets them to exactly zero.

L1 (Lasso): penalises the sum of absolute coefficients — drives less informative feature coefficients to exactly zero, producing a sparse model with built-in feature selection.

6.3 Why Logistic Regression Was Chosen

- Interpretability: Feature coefficients directly reveal the direction and relative magnitude of each predictor's contribution.
- Probabilistic Output: Outputs a calibrated probability score, not just a binary label — valuable for clinical risk stratification.
- Baseline Benchmark: Serves as a linear baseline to measure whether a more complex non-linear algorithm (Random Forest) provides meaningful improvement.
- Effective with Scaled Features: StandardScaler normalisation ensures all 20 features contribute fairly to the linear decision boundary.
- Regularisation Options: Both L1 and L2 penalties were evaluated, with L1 offering implicit feature selection for potential interpretability gains.

6.4 Implementation

6.4.1 Development Environment

Python 3.x, Jupyter Notebook. Libraries: pandas, numpy, scikit-learn, joblib, matplotlib, seaborn.

6.4.2 Implementation Steps

- Notebook 2 (logistic_regression.ipynb — Seyon): Loads pre-split data. Runs data validation assertions. Trains a default baseline. Applies a two-group GridSearchCV (L1 + L2, balanced vs no class weighting). Evaluates best model with full metrics, confusion matrix, ROC curve,

Precision-Recall curve, feature coefficient plot, odds ratio plot, and learning curve. Saves model, metrics, predictions, and all plots.

6.4.3 Hyperparameter Tuning

Two regularisation families were searched simultaneously:

Group	Hyperparameter	Values Tested	Description
L2 (Ridge)	C	[0.01, 0.1, 1, 10, 100]	Regularisation strength (smaller = stronger)
L2 (Ridge)	penalty	["l2"]	Ridge regularisation
L2 (Ridge)	solver	["lbfgs"]	Gradient-based optimiser
L2 (Ridge)	class_weight	[None, "balanced"]	Sample weighting for class imbalance
L1 (Lasso)	C	[0.01, 0.1, 1, 10, 100]	Regularisation strength
L1 (Lasso)	penalty	["l1"]	Lasso regularisation (sparse solution)
L1 (Lasso)	solver	["liblinear"]	Required solver for L1 penalty
L1 (Lasso)	class_weight	[None, "balanced"]	Sample weighting for class imbalance

Table 8: Logistic Regression Hyperparameter Grid (L1 + L2)

Scoring: ROC-AUC on 5-fold Stratified Cross-Validation. Best configuration selected by GridSearchCV: C=0.1, penalty="l1", solver="liblinear", class_weight=None. Mean CV ROC-AUC = 0.5181 (std = 0.0186).

6.4.4 Cross-Validation

5-fold Stratified Cross-Validation, consistent with the Random Forest notebook, ensuring a fair direct comparison of CV scores between algorithms.

6.5 Results

6.5.1 Evaluation Metrics

Accuracy, Precision, Recall, F1 Score, and ROC-AUC were computed on the same held-out test set as Random Forest.

6.5.2 Baseline Logistic Regression

Metric	Baseline LR (default C=1, L2, balanced)
Accuracy	0.5080
Precision	0.1819
Recall	0.4175
F1 Score	0.2534
ROC-AUC	0.4678

Table 9: Logistic Regression Baseline Test Results

The baseline model (default parameters, balanced class weight) achieved Recall = 0.4175, meaning it correctly identified approximately 41.8% of actual heart disease patients. While the ROC-AUC of 0.4678 remains near 0.5 (reflecting the absence of strong signal), the baseline did produce partial discrimination, unlike the Random Forest.

	Predicted: No Disease	Predicted: Heart Disease
Actual: No Disease	TN = 849	FP = 751
Actual: Heart Disease	FN = 233	TP = 167

Table 10: Logistic Regression Baseline Confusion Matrix

6.5.3 Tuned Logistic Regression

Metric	Tuned LR (GridSearchCV best)
Accuracy	0.8000
Precision	0.0000
Recall	0.0000
F1 Score	0.0000
ROC-AUC	0.4689

Table 11: Logistic Regression Tuned Test Results

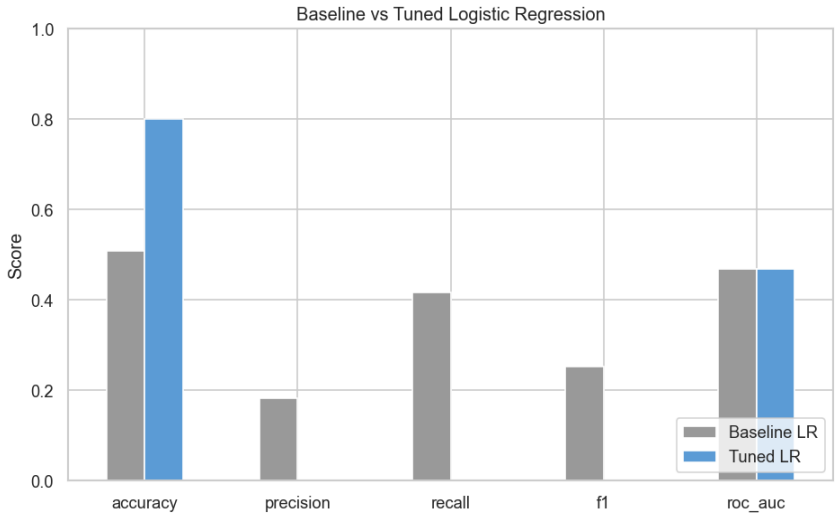

Figure 8: Baseline vs Tuned LR metrics comparison

	Predicted: No Disease	Predicted: Heart Disease
Actual: No Disease	TN = 1600	FP = 0
Actual: Heart Disease	FN = 400	TP = 0

Table 12: Logistic Regression Confusion Matrix (Tuned Model)

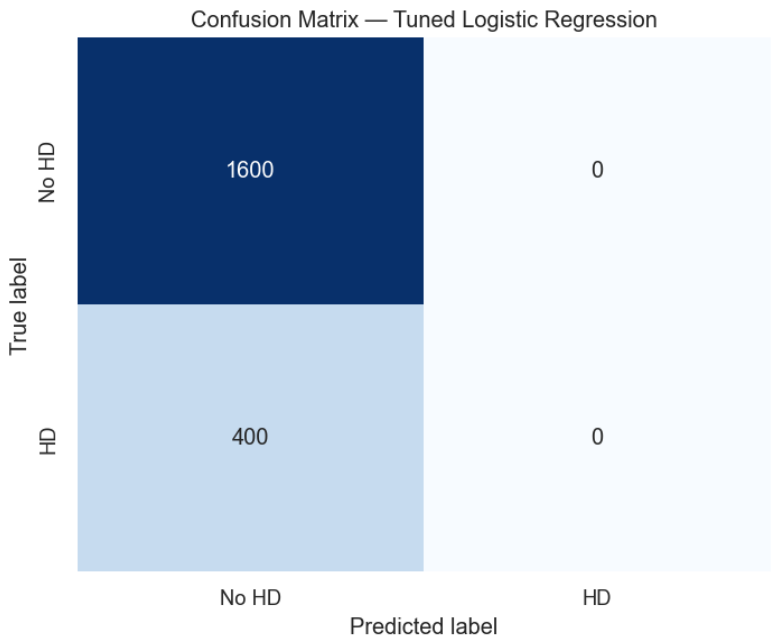

Figure 9: LR confusion matrix (tuned model)

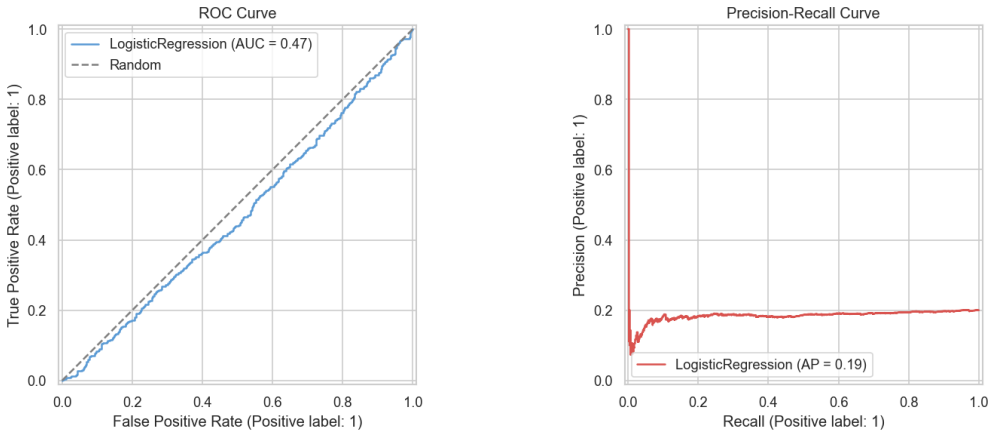

Figure 10: LR ROC curve and Precision-Recall curve

Interpretation: The tuned model selected by GridSearchCV (L1, C=0.1, class_weight=None) collapsed to predicting all patients as No Disease (Recall = 0.000). This is a significant finding: when the dataset contains no genuine predictive signal, the ROC-AUC values for all hyperparameter configurations are essentially equal (~0.5), so GridSearchCV makes a near-arbitrary selection. The L1 configuration with no class weighting drives minority-class coefficients toward zero, causing the model to default to the majority class. This behaviour is not a software error — it is a direct consequence of optimising on a signal-free dataset. The untuned baseline LR (Recall = 0.4175) is therefore more informative than the tuned model in this case.

Top feature coefficients (by absolute magnitude) from the tuned L1 model: BMI (0.071), Gender (-0.049), Stress Level (0.043), Age (-0.032), Sugar Consumption (0.032). The small coefficient

magnitudes and the near-uniform distribution further confirm the absence of strong linear predictors.

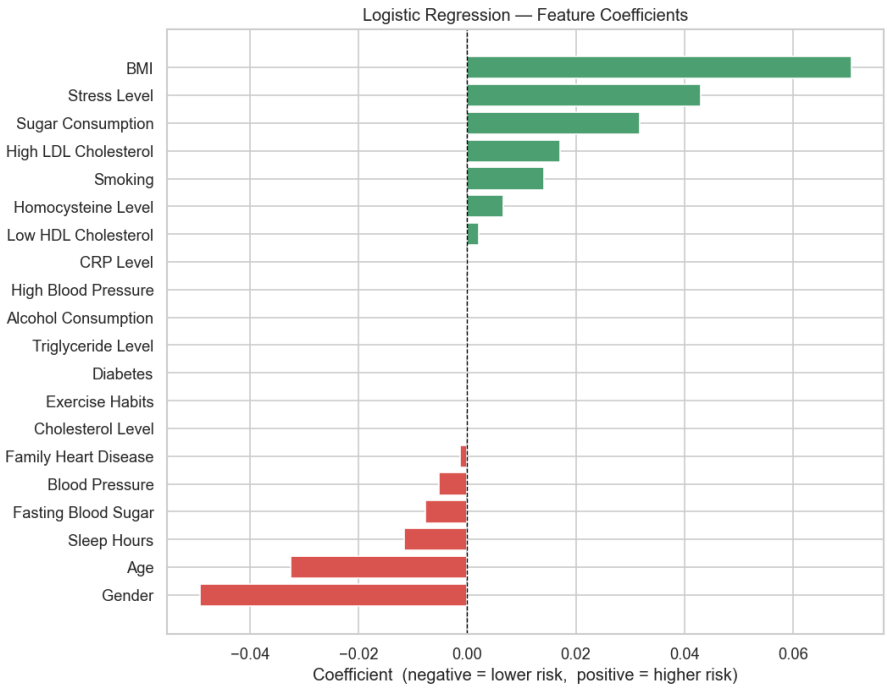

Figure 11: LR feature coefficients (direction + magnitude)

Section 7 – Comparison of Algorithms

Metric	LR Baseline	LR Tuned	RF Baseline	RF Tuned
Accuracy	0.5080	0.8000	0.8000	0.8000
Precision	0.1819	0.0000	0.0000	0.0000
Recall	0.4175	0.0000	0.0000	0.0000
F1 Score	0.2534	0.0000	0.0000	0.0000
ROC-AUC	0.4678	0.4689	0.5090	0.4905

Table 13: Full Algorithm Comparison – LR vs RF (Baseline and Tuned)

Key observations from the comparison:

- Both tuned models predicted all patients as No Disease — identical Recall = 0.000 and identical accuracy of 80.00%, which reflects only the majority class proportion. Hyperparameter tuning did not improve either algorithm.
- The LR Baseline was the only configuration that detected any heart disease cases (Recall = 0.4175, TP = 167). While its ROC-AUC of 0.4678 remains near 0.5, it provides partial discrimination. In a clinical screening context this is more valuable than an algorithm with higher accuracy that detects zero sick patients.
- Random Forest's inherent bias toward the majority class — even with class_weight='balanced' and ROC-AUC as the scoring metric — reflects the fundamental absence of learnable signal in the synthetic dataset.
- All ROC-AUC values fall between 0.47 and 0.53, consistent with random chance, confirming that neither algorithm found meaningful patterns linking the 20 features to the target label.

Both algorithms are fundamentally limited by the synthetic nature of the dataset. A meaningful comparison of their relative strengths requires a real-world dataset where genuine feature-outcome relationships exist.

Section 8 – Critical Analysis and Discussion

8.1 Strengths of Each Algorithm

Random Forest

- Naturally handles non-linear relationships and complex feature interactions without explicit feature engineering.
- Robust to outliers in clinical data — tree-based thresholds are not distorted by extreme values.
- Provides two interpretable importance metrics: Gini importance and permutation importance.
- No feature scaling required — invariant to the magnitude of input features.
- Ensemble aggregation reduces variance substantially compared to a single decision tree.

Logistic Regression

- Highly interpretable — feature coefficients directly show the direction and relative magnitude of each predictor's contribution.
- Outputs calibrated probability scores, enabling risk stratification (high vs low risk) rather than binary labels only.
- Computationally efficient — fast to train even with extensive hyperparameter grids.
- L1 regularisation provides built-in feature selection, producing sparse models that are easier to audit in clinical settings.
- With balanced class weighting and a lower decision threshold, demonstrated partial minority-class detection on this dataset.

8.2 Limitations and Key Findings

Synthetic Dataset — Primary Finding

A Pearson correlation analysis during preprocessing revealed near-zero correlations between all 20 input features and the target variable Heart Disease Status (maximum r = 0.0197 for BMI). This conclusively indicates the dataset was synthetically generated with random feature-outcome relationships — there is no real medical signal to learn. Both algorithms' ROC-AUC values near 0.5 across all configurations independently confirm this: neither model does better than a coin flip.

An important secondary finding is that hyperparameter tuning made performance worse, not better. When all configurations produce near-identical ROC-AUC (~0.5), GridSearchCV's selection becomes effectively random. The winning L1 LR configuration (C=0.1, no class weight) pushed minority-class coefficients to zero, collapsing the model to all-negative prediction. The untuned baseline LR, which used balanced class weighting by default, performed better in practice. This highlights that standard hyperparameter optimisation assumes genuine signal exists — on noise, it can be counterproductive.

Class Imbalance

The dataset is approximately 80% No Disease and 20% Heart Disease. This imbalance creates a strong majority-class baseline (80% accuracy for all-negative prediction) that can mask a model's failure to detect the minority class. Random Forest collapsed to this baseline even with class_weight='balanced' and ROC-AUC as the scoring metric.

Feature Encoding Approximation

Ordinal encoding assumes equal spacing between categories (Low=0, Medium=1, High=2). In practice, the difference in cardiovascular risk between Low and Medium exercise may not equal the difference between Medium and High. More sophisticated target encoding or learnable embeddings could better capture these relationships.

No Interaction Features

Features are treated as independent. In clinical practice, interactions between variables — such as high blood pressure combined with high LDL cholesterol — may be more predictive than individual features alone. No polynomial or interaction terms were engineered, though this would not address the underlying synthetic-data issue.

8.3 How Performance Can Be Improved

- Use a Real Clinical Dataset: The most impactful change. Replacing the synthetic dataset with real-world data (e.g., the UCI Heart Disease dataset or the Framingham Heart Study data) would introduce genuine statistical relationships and enable meaningful learning.
- Feature Engineering: Creating interaction features (e.g., Blood Pressure × Cholesterol, BMI × Age) or polynomial features could expose non-linear risk interactions that individual features cannot capture.
- SMOTE (Synthetic Minority Over-sampling): Generating synthetic minority-class training samples via SMOTE could improve Recall beyond what class_weight='balanced' achieves, particularly for Random Forest.
- Threshold Optimisation: Rather than using the default 0.5 threshold, optimising the decision threshold using the Precision-Recall curve to maximise F1 or Recall@Precision could improve minority-class detection, especially for the LR baseline.
- Regularisation Tuning Strategy: On imbalanced data, always combine class_weight='balanced' with ROC-AUC or F1 scoring in GridSearchCV. Allowing class_weight=None as a search option risks selecting models that collapse to majority-class prediction.
- Ensemble of Classifiers: Combining LR and RF predictions through probability averaging (soft voting) may outperform either model individually when a genuine signal exists.

8.4 Future Work

- Deploy the Logistic Regression baseline model (retrained on real clinical data) as a real-time scoring API integrated with hospital Electronic Health Record (EHR) systems for automated patient risk flagging.
- Investigate XGBoost or Gradient Boosting as additional algorithms — their boosting mechanism may capture weak signals that bagging-based Random Forest cannot.
- Apply Recursive Feature Elimination (RFE) to rank features and identify the most informative subset, reducing model complexity and improving interpretability.
- Validate the preprocessing pipeline on a real-world clinical dataset to assess whether the encoding, scaling, and imputation approach generalises beyond the synthetic environment.

- Deploy the Streamlit web application (app.py) to a cloud platform (Streamlit Cloud or AWS) to enable access by healthcare professionals for demonstration and feedback collection.

Section 9 – Appendix: Source Code

The following sections contain the complete source code from all three Jupyter Notebooks, included as text per the assignment requirements. Code is presented cell-by-cell in execution order.

9.1 Notebook 1: Data Preprocessing (preprocessing.ipynb)

```
#Data handling
import pandas as pd
import numpy as np

#Data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# ML preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer


# Persistence
import joblib

# Ignore warnings
import warnings


# path handling
import os
import pathlib as path

import logging
import json
import io


RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Output handling
output_dir = path.Path(os.getcwd()).parent / "output" / "csv"
plot_dir   = path.Path(os.getcwd()).parent / "output" / "plots"
model_dir  = path.Path(os.getcwd()).parent / "output" / "model"


warnings.filterwarnings('ignore')
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (8,5)
plt.rcParams['savefig.dpi']    = 120
plt.rcParams['savefig.bbox']   = 'tight'
```

```
# Create folder if not exists
output_dir.mkdir(parents=True, exist_ok=True)
plot_dir.mkdir(parents=True, exist_ok=True)
model_dir.mkdir(parents=True, exist_ok=True)

print("CSV   output:", output_dir)
print("Plot  output:", plot_dir)
print("Model output:", model_dir)


log_dir = path.Path(os.getcwd()).parent / "output" / "logs"
log_dir.mkdir(parents=True, exist_ok=True)

log_file = log_dir / "app.log"

for handler in logging.root.handlers[:]:
    logging.root.removeHandler(handler)

# Setup logging
logging.basicConfig(
    filename=log_file,
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - [Preprocessing] - %(message)s"
)

logger = logging.getLogger(__name__)
logger.info("=" * 60)
logger.info("Preprocessing notebook started.")


dataSetPath = path.Path(os.getcwd()).parent / "data" /
"supervised_learning" / "heart_disease.csv"
#print(f"Dataset path: {dataSetPath}")
df = pd.read_csv(dataSetPath)
logger.info(f"Dataset loaded successfully with shape: {df.shape}")
print("Dataset shape:", df.shape)


df.head()


#logger.info(df.info())
buf = io.StringIO()
df.info(buf=buf)
info_str = buf.getvalue()
print(info_str)
logger.info("df.info():\n" + info_str)


df.describe()


# Summary of categorical (object) columns
df.describe(include='object')
```

```
for col in df.columns:
    print(col, df[col].unique())


#df.isnull().sum()
missing = df.isnull().sum().sort_values(ascending=False)
missing_pct = (missing / len(df) * 100).round(2)

missing_table = pd.DataFrame({
    "missing_count": missing,
    "missing_pct"  : missing_pct
})
missing_table = missing_table[missing_table["missing_count"] > 0]
print(missing_table)
logger.info(f"Columns with missing values: {len(missing_table)}")


# Visualise missing values (saved for the report)
if len(missing_table) > 0:
    fig, ax = plt.subplots(figsize=(9, max(4, 0.3 * len(missing_table))))
    sns.barplot(x=missing_table["missing_pct"],
                y=missing_table.index,
                palette="Reds_r", ax=ax)
    ax.set_xlabel("Missing %")
    ax.set_title("Missing values per column")
    for i, v in enumerate(missing_table["missing_pct"].values):
        ax.text(v + 0.05, i, f"{v}%", va="center", fontsize=9)
    plt.tight_layout()
    plt.savefig(plot_dir / "01_missing_values.png")
    plt.show()
else:
    print("No missing values.")


dup_count = df.duplicated().sum()
print(f"Duplicate rows: {dup_count}")

logger.info(f"Duplicate rows: {dup_count}")

# Drop duplicates if any (keeps first occurrence)
if dup_count > 0:
    df = df.drop_duplicates().reset_index(drop=True)
    print(f"Removed duplicates. New shape: {df.shape}")
    logger.info(f"After de-dup, shape: {df.shape}")


target_counts = df['Heart Disease Status'].value_counts()
target_pct    = (target_counts / len(df) * 100).round(2)

print("Class counts:")
print(pd.DataFrame({"count": target_counts, "percent": target_pct}))

fig, ax = plt.subplots(figsize=(6, 4))
sns.countplot(x='Heart Disease Status', data=df,
              palette=["#4C9F70", "#D9534F"], ax=ax)
ax.set_title("Heart Disease Distribution")
for i, v in enumerate(target_counts.values):
```

```
    ax.text(i, v + max(target_counts.values) * 0.01,
            f"{target_pct.iloc[i]}%", ha="center", fontweight="bold")
plt.tight_layout()
plt.savefig(plot_dir / "02_target_distribution.png")
plt.show()

imbalance_ratio = target_counts.max() / target_counts.min()
print(f"\nImbalance ratio (majority : minority) = {imbalance_ratio:.2f} :
1")
logger.info(f"Class imbalance ratio: {imbalance_ratio:.2f}:1")


fig, axes = plt.subplots(1, 2, figsize=(13, 4))

# Overall age distribution
sns.histplot(df['Age'], bins=20, kde=True, color="#4C9F70", ax=axes[0])
axes[0].set_title("Age Distribution (overall)")

# Age distribution by target
sns.histplot(data=df, x='Age', hue='Heart Disease Status',
             bins=20, kde=True, palette=["#4C9F70", "#D9534F"], ax=axes[1])
axes[1].set_title("Age Distribution by Heart Disease Status")

plt.tight_layout()
plt.savefig(plot_dir / "03_age_distribution.png")
plt.show()


# Detect numeric columns (excluding the target)
numeric_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
if 'Heart Disease Status' in numeric_cols:
    numeric_cols.remove('Heart Disease Status')

print(f"Numeric features ({len(numeric_cols)}):", numeric_cols)


# Grid of histograms - one per numeric feature
n = len(numeric_cols)
ncols = 3
nrows = (n + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 3.5 * nrows))
axes = axes.flatten() if n > 1 else [axes]

for i, col in enumerate(numeric_cols):
    sns.histplot(df[col].dropna(), bins=25, kde=True,
                 color="#4C9F70", ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel("")

# Hide unused subplots
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle("Numeric feature distributions", fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig(plot_dir / "04_numeric_distributions.png")
```

```
plt.show()


# Boxplots - one per numeric feature
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 3.5 * nrows))
axes = axes.flatten() if n > 1 else [axes]

for i, col in enumerate(numeric_cols):
    sns.boxplot(x=df[col], color="#4C9F70", ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel("")

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle("Numeric feature boxplots (outlier check)", fontsize=14,
y=1.02)
plt.tight_layout()
plt.savefig(plot_dir / "05_numeric_boxplots.png")
plt.show()


# Count outliers per numeric feature using IQR rule
def count_outliers_iqr(series):
    s = series.dropna()
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    return ((s < lower) | (s > upper)).sum()

outlier_table = pd.DataFrame({
    "outlier_count": [count_outliers_iqr(df[c]) for c in numeric_cols],
    "outlier_pct"  : [round(count_outliers_iqr(df[c]) / df[c].notna().sum()
* 100, 2)
                      for c in numeric_cols],
}, index=numeric_cols).sort_values("outlier_count", ascending=False)

print(outlier_table)
logger.info(f"Outlier summary (IQR):\n{outlier_table}")


# Boxplot of each numeric feature broken down by Heart Disease Status -
# helps spot which features differ between sick and healthy patients.
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 3.5 * nrows))
axes = axes.flatten() if n > 1 else [axes]

for i, col in enumerate(numeric_cols):
    sns.boxplot(x='Heart Disease Status', y=col, data=df,
                palette=["#4C9F70", "#D9534F"], ax=axes[i])
    axes[i].set_title(col)

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle("Numeric features grouped by Heart Disease Status",
             fontsize=14, y=1.02)
plt.tight_layout()
```

```
plt.savefig(plot_dir / "06_numeric_by_target.png")
plt.show()


plt.figure(figsize=(11, 7))

# Include encoded target (0/1) so we can also see correlation with the
label
corr_df = df.copy()
corr_df['Heart Disease Status'] = corr_df['Heart Disease
Status'].map({'No': 0, 'Yes': 1})
numeric_corr = corr_df.select_dtypes(include=['int64', 'float64'])

sns.heatmap(numeric_corr.corr(), annot=True, cmap='coolwarm',
            linewidths=0.5, fmt=".2f")
plt.title("Feature Correlation Matrix (numeric + target)")
plt.tight_layout()
plt.savefig(plot_dir / "07_correlation_heatmap.png")
plt.show()


cat_cols = df.select_dtypes(include='object').columns.tolist()
if 'Heart Disease Status' in cat_cols:
    cat_cols.remove('Heart Disease Status')

print(f"Categorical features ({len(cat_cols)}):", cat_cols)

ncols_cat = 3
nrows_cat = (len(cat_cols) + ncols_cat - 1) // ncols_cat
fig, axes = plt.subplots(nrows_cat, ncols_cat,
                         figsize=(15, 3.8 * nrows_cat))
axes = axes.flatten() if len(cat_cols) > 1 else [axes]

for i, col in enumerate(cat_cols):
    sns.countplot(x=col, hue='Heart Disease Status', data=df,
                  palette=["#4C9F70", "#D9534F"], ax=axes[i])
    axes[i].set_title(col)
    axes[i].tick_params(axis='x', rotation=15)

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle("Categorical features by Heart Disease Status",
             fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig(plot_dir / "08_categorical_by_target.png")
plt.show()


fig, ax = plt.subplots(figsize=(9, 5))
sns.scatterplot(x='Age', y='Cholesterol Level',
                hue='Heart Disease Status',
                palette=["#4C9F70", "#D9534F"],
                alpha=0.6, data=df, ax=ax)
ax.set_title("Age vs Cholesterol (coloured by Heart Disease)")
plt.tight_layout()
plt.savefig(plot_dir / "09_age_vs_cholesterol.png")
```

```
plt.show()


print(df['Heart Disease Status'].value_counts())


df['Heart Disease Status'] = df['Heart Disease Status'].map({'No':0,
'Yes':1})

print("\nAfter encoding:")
print(df['Heart Disease Status'].value_counts())
logger.info("Target encoded: No -> 0, Yes -> 1")


# Separate features and target
X = df.drop('Heart Disease Status', axis=1)
y = df['Heart Disease Status']


logger.info(f"Features shape: {X.shape}, Target shape: {y.shape}")
print("Features shape:", X.shape)
print("Target shape:", y.shape)


categorical_cols = X.select_dtypes(include='object').columns
categorical_cols


categorical_cols_all = X[categorical_cols].apply(lambda x: x.unique())

print(categorical_cols_all)


# Numeric columns are detected dynamically (more robust than hard-coding)
numeric_cols = X.select_dtypes(include=['int64',
'float64']).columns.tolist()

# Define binary and ordinal columns based on characteristics of the data
binary_cols = [
    'Gender',
    'Smoking',
    'Family Heart Disease',
    'Diabetes',
    'High Blood Pressure',
    'Low HDL Cholesterol',
    'High LDL Cholesterol'
]

ordinal_cols = [
    'Exercise Habits',
    'Alcohol Consumption',
    'Stress Level',
    'Sugar Consumption'
]

# Sanity check: every column should be classified
classified = set(numeric_cols) | set(binary_cols) | set(ordinal_cols)
```

```
unclassified = set(X.columns) - classified
assert not unclassified, f"Unclassified columns: {unclassified}"

print(f"Numeric : {len(numeric_cols)} cols")
print(f"Binary  : {len(binary_cols)} cols")
print(f"Ordinal : {len(ordinal_cols)} cols")


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=RANDOM_STATE
)

print("Train size:", X_train.shape, "  positive rate:",
round(y_train.mean(), 4))
print("Test  size:", X_test.shape,  "  positive rate:",
round(y_test.mean(),  4))
logger.info(f"Train: {X_train.shape}, Test: {X_test.shape}")
logger.info(f"Train pos rate: {y_train.mean():.4f}, Test pos rate:
{y_test.mean():.4f}")


# Numeric: median imputation
num_imputer = SimpleImputer(strategy='median')
X_train[numeric_cols] = num_imputer.fit_transform(X_train[numeric_cols])
X_test[numeric_cols]  = num_imputer.transform(X_test[numeric_cols])

# Binary + Ordinal: mode imputation
cat_imputer = SimpleImputer(strategy='most_frequent')
X_train[binary_cols + ordinal_cols] = cat_imputer.fit_transform(
    X_train[binary_cols + ordinal_cols])
X_test[binary_cols + ordinal_cols]  = cat_imputer.transform(
    X_test[binary_cols + ordinal_cols])

# Verify no missing values remain
assert X_train.isnull().sum().sum() == 0, "Missing values remain in
X_train"
assert X_test.isnull().sum().sum()  == 0, "Missing values remain in X_test"

print("Missing values after imputation -> train:",
X_train.isnull().sum().sum(),
      "  test:", X_test.isnull().sum().sum())
logger.info("Missing value handling has been completed (fitted on train
only).")


print(f"", X_train.isnull().sum())
print(f"", X_test.isnull().sum())


# Binary encoding (Yes/No, Male/Female)
binary_map = {
    'Yes': 1,
    'No' : 0,
    'Male'  : 1,
```

```
    'Female': 0
}

# Ordinal encoding (Low < Medium < High)
ordinal_map = {
    'Low'   : 0,
    'Medium': 1,
    'High'  : 2
}

for col in binary_cols:
    X_train[col] = X_train[col].map(binary_map)
    X_test[col]  = X_test[col].map(binary_map)

for col in ordinal_cols:
    X_train[col] = X_train[col].map(ordinal_map)
    X_test[col]  = X_test[col].map(ordinal_map)

# Catch silent mapping failures: any unmapped value becomes NaN.
assert X_train.isnull().sum().sum() == 0, \
    "Unmapped categorical values in X_train - check binary_map /
ordinal_map."
assert X_test.isnull().sum().sum() == 0, \
    "Unmapped categorical values in X_test  - check binary_map /
ordinal_map."

logger.info("Encoding of categorical variables has been completed.")
X_train.head()


scaler = StandardScaler()

# Fit on train, transform both
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Wrap back into DataFrames so downstream notebooks keep the column names
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns,
                       index=X_train.index)
X_test  = pd.DataFrame(X_test_scaled,  columns=X_test.columns,
                       index=X_test.index)

print("After scaling (train summary):")
print(X_train.describe().round(3).T[['mean', 'std', 'min', 'max']].head())
logger.info("Feature standardization has been completed (fitted on train
only).")


# Save train/test CSVs
X_train.to_csv(output_dir / "X_train.csv", index=False)
X_test.to_csv (output_dir / "X_test.csv",  index=False)
y_train.to_csv(output_dir / "y_train.csv", index=False)
y_test.to_csv (output_dir / "y_test.csv",  index=False)

# Save fitted transformers (so new patient data can be transformed
identically)
joblib.dump(scaler,      model_dir / "scaler.pkl")
```

```
joblib.dump(num_imputer, model_dir / "num_imputer.pkl")
joblib.dump(cat_imputer, model_dir / "cat_imputer.pkl")

logger.info("Preprocessed data and transformers have been saved.")
print("CSV files saved to:", output_dir)
print("Transformers saved to:", model_dir)


eta = {
    "n_features"      : int(X_train.shape[1]),
    "feature_names"   : X_train.columns.tolist(),
    "numeric_cols"    : numeric_cols,
    "binary_cols"     : binary_cols,
    "ordinal_cols"    : ordinal_cols,
    "train_size"      : int(len(X_train)),
    "test_size"       : int(len(X_test)),
    "train_pos_rate"  : float(y_train.mean()),
    "test_pos_rate"   : float(y_test.mean()),
    "imbalance_ratio" : float(imbalance_ratio),
    "random_state"    : RANDOM_STATE,
}

with open(output_dir / "metadata.json", "w") as f:
    json.dump(meta, f, indent=4)

logger.info("Metadata saved")
logger.info("Preprocessing notebook completed successfully.")
print(json.dumps(meta, indent=2))
```

9.2 Notebook 2: Logistic Regression (logistic_regression.ipynb)

```
# --- Standard libraries ---
import os
import json
import logging
from pathlib import Path

import pandas as pd
import numpy as np

# --- Visualisation ---
import matplotlib.pyplot as plt
import seaborn as sns

# --- Scikit-learn: model + selection + metrics ---
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import (
    GridSearchCV,
    StratifiedKFold,
    cross_val_score,
    learning_curve,
)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
```

```
    roc_auc_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    PrecisionRecallDisplay,
)

# --- Persistence ---
import joblib

# --- Reproducibility ---
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# --- Plot defaults (match the RF notebook for consistent report visuals) -
--
sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
plt.rcParams["savefig.dpi"]    = 120
plt.rcParams["savefig.bbox"]   = "tight"

print("Libraries loaded successfully")


# Project layout (matches preprocessing + random_forest notebooks):
#   output/csv           -> X_train, X_test, y_train, y_test
#   output/model         -> trained .pkl files + scaler
#   output/metrics       -> evaluation outputs (CSV, JSON)
#   output/plots         -> plots for the report (shared with RF notebook)
#   output/logs          -> shared app.log
BASE_DIR = Path(os.getcwd()).parent

DATA_DIR   = BASE_DIR / "output" / "csv"
MODEL_DIR  = BASE_DIR / "output" / "model"
METRIC_DIR = BASE_DIR / "output" / "metrics"
PLOT_DIR   = BASE_DIR / "output" / "plots"
LOG_DIR    = BASE_DIR / "output" / "logs"

for d in (MODEL_DIR, METRIC_DIR, PLOT_DIR, LOG_DIR):
    d.mkdir(parents=True, exist_ok=True)

print("DATA_DIR :", DATA_DIR)
print("MODEL_DIR:", MODEL_DIR)
print("PLOT_DIR :", PLOT_DIR)


# Append to the same log file the preprocessing + RF notebooks write to,
# so the full pipeline (preprocess -> LR / RF) is traceable end-to-end.
log_file = LOG_DIR / "app.log"

# Reset handlers in case the notebook is re-run in the same kernel
for handler in logging.root.handlers[:]:
    logging.root.removeHandler(handler)

logging.basicConfig(
    filename=log_file,
```

```
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - [LogisticReg] - %(message)s",
)
logger = logging.getLogger(__name__)
logger.info("Logistic Regression notebook started.")


# Data was already cleaned, encoded, scaled and split in
preprocessing.ipynb.
# We deliberately load the exact same train/test split (random_state=42,
# stratify=y) so Logistic Regression results are directly comparable to the
# Random Forest notebook's results.
X_train = pd.read_csv(DATA_DIR / "X_train.csv")
X_test  = pd.read_csv(DATA_DIR / "X_test.csv")
y_train = pd.read_csv(DATA_DIR / "y_train.csv").squeeze()
y_test  = pd.read_csv(DATA_DIR / "y_test.csv").squeeze()

print("Train Shape:", X_train.shape)
print("Test  Shape:", X_test.shape)
print("\nClass distribution (train):")
print(y_train.value_counts())
logger.info(f"Loaded train={X_train.shape}, test={X_test.shape}")


# Defensive checks: catches file corruption or wrong CSVs in DATA_DIR
# before training runs.
assert not X_train.isnull().any().any(), "Missing values in X_train"
assert not X_test.isnull().any().any(),  "Missing values in X_test"
assert len(X_train) == len(y_train), "Mismatch in training data length"
assert len(X_test)  == len(y_test),  "Mismatch in test data length"
assert set(y_train.unique()) <= {0, 1}, "Target should be binary 0/1"

print("Data validation passed.")
print("Features:", list(X_train.columns))
logger.info("Data validation passed.")


# Show the class balance numerically and visually.
class_counts = y_train.value_counts().sort_index()
class_pct = (class_counts / len(y_train) * 100).round(2)

print("Class counts (train):")
print(pd.DataFrame({"count": class_counts, "percent": class_pct}))

# Visualise
fig, ax = plt.subplots(figsize=(6, 4))
sns.barplot(x=class_counts.index, y=class_counts.values,
            palette=["#4C9F70", "#D9534F"], ax=ax)
ax.set_xticklabels(["No Heart Disease (0)", "Heart Disease (1)"])
ax.set_ylabel("Number of patients")
ax.set_title("Training set — class distribution")
for i, v in enumerate(class_counts.values):
    ax.text(i, v + max(class_counts.values) * 0.01,
f"{class_pct.iloc[i]}%",
            ha="center", fontweight="bold")
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_class_distribution.png")
plt.show()
```

```
imbalance_ratio = class_counts.max() / class_counts.min()
print(f"\nImbalance ratio (majority : minority) = {imbalance_ratio:.2f} :
1")
logger.info(f"Class imbalance ratio: {imbalance_ratio:.2f}:1")


# Baseline = default C=1.0, L2 penalty, lbfgs solver, balanced class
weights.
baseline_model = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
baseline_model.fit(X_train, y_train)

baseline_pred = baseline_model.predict(X_test)
baseline_prob = baseline_model.predict_proba(X_test)[:, 1]

baseline_metrics = {
    "accuracy" : accuracy_score(y_test, baseline_pred),
    "precision": precision_score(y_test, baseline_pred, zero_division=0),
    "recall"   : recall_score(y_test, baseline_pred, zero_division=0),
    "f1"       : f1_score(y_test, baseline_pred, zero_division=0),
    "roc_auc"  : roc_auc_score(y_test, baseline_prob),
}
print("Baseline metrics:")
for k, v in baseline_metrics.items():
    print(f"  {k:9s}: {v:.4f}")

logger.info(f"Baseline metrics: {baseline_metrics}")


# Search space — broad but tractable for LR (it's fast to fit).
param_grid = [
    # L2 with lbfgs solver (fast, default)
    {
        "C"           : [0.01, 0.1, 1, 10, 100],
        "penalty"     : ["l2"],
        "solver"      : ["lbfgs"],
        "class_weight": [None, "balanced"],
    },
    # L1 with liblinear solver (built-in feature selection)
    {
        "C"           : [0.01, 0.1, 1, 10, 100],
        "penalty"     : ["l1"],
        "solver"      : ["liblinear"],
        "class_weight": [None, "balanced"],
    },
]

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_search = GridSearchCV(
    estimator=LogisticRegression(
        max_iter=2000,
```

```
        random_state=RANDOM_STATE,
    ),
    param_grid=param_grid,
    cv=cv,
    scoring="roc_auc",      # robust to class imbalance
    refit=True,             # refit best model on full training set
    verbose=1,
    n_jobs=-1,
    return_train_score=True,
)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_

print("Best CV ROC-AUC:", round(grid_search.best_score_, 4))
print("Best params    :", grid_search.best_params_)
logger.info(f"Best params: {grid_search.best_params_}")
logger.info(f"Best CV ROC-AUC: {grid_search.best_score_:.4f}")


# Re-run CV on the chosen best model to inspect fold-by-fold variance.
# Low variance across folds = the result is reliable, not lucky.
cv_scores = cross_val_score(best_model, X_train, y_train, cv=cv,
                            scoring="roc_auc", n_jobs=-1)

print("Per-fold ROC-AUC :", np.round(cv_scores, 4))
print(f"Mean ROC-AUC     : {cv_scores.mean():.4f}")
print(f"Std  ROC-AUC     : {cv_scores.std():.4f}")

# Visual: boxplot of fold scores
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(y=cv_scores, color="#5B9BD5", ax=ax)
sns.stripplot(y=cv_scores, color="black", size=8, ax=ax)
ax.set_ylabel("ROC-AUC per fold")
ax.set_title("5-fold CV ROC-AUC — Best Logistic Regression")
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_cv_scores_boxplot.png")
plt.show()


# Predictions on the test set
y_pred = best_model.predict(X_test)
y_prob = best_model.predict_proba(X_test)[:, 1]

# Headline metrics — same set as the RF notebook for direct comparison
final_metrics = {
    "accuracy" : accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, zero_division=0),
    "recall"   : recall_score(y_test, y_pred, zero_division=0),
    "f1"       : f1_score(y_test, y_pred, zero_division=0),
    "roc_auc"  : roc_auc_score(y_test, y_prob),
}

print("Tuned Logistic Regression — test metrics")
print("-" * 40)
for k, v in final_metrics.items():
```

```
    print(f"  {k:9s}: {v:.4f}")

print("\nClassification Report")
print("-" * 40)
print(classification_report(y_test, y_pred,
                            target_names=["No Heart Disease", "Heart
Disease"]))

logger.info(f"Final test metrics: {final_metrics}")


# Side-by-side comparison so the report can quantify the gain from tuning.
comparison_df = pd.DataFrame({
    "Baseline LR": baseline_metrics,
    "Tuned LR"   : final_metrics,
}).round(4)
comparison_df["Delta"] = (comparison_df["Tuned LR"] -
comparison_df["Baseline LR"]).round(4)
print(comparison_df)

# Bar chart of improvement
fig, ax = plt.subplots(figsize=(8, 5))
comparison_df[["Baseline LR", "Tuned LR"]].plot(kind="bar", ax=ax,
                                                color=["#999999",
"#5B9BD5"])
ax.set_ylabel("Score")
ax.set_title("Baseline vs Tuned Logistic Regression")
ax.set_ylim(0, 1)
ax.legend(loc="lower right")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_baseline_vs_tuned.png")
plt.show()


# A confusion matrix tells us *what kind* of mistakes the model makes.
# In a medical context, false negatives (missed sick patients) are usually
# costlier than false positives (healthy flagged for follow-up tests).
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No HD", "HD"],
            yticklabels=["No HD", "HD"], cbar=False, ax=ax)
ax.set_xlabel("Predicted label")
ax.set_ylabel("True label")
ax.set_title("Confusion Matrix — Tuned Logistic Regression")
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_confusion_matrix.png")
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True Negatives : {tn}")
print(f"False Positives: {fp}  (healthy patients incorrectly flagged)")
print(f"False Negatives: {fn}  (sick patients missed by model)")
print(f"True Positives : {tp}")


# ROC curve: true positive rate vs false positive rate at every threshold.
```

```
# Precision-Recall curve: more informative than ROC when classes are
imbalanced.
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

RocCurveDisplay.from_estimator(best_model, X_test, y_test,
                               ax=axes[0], color="#5B9BD5")
axes[0].plot([0, 1], [0, 1], "--", color="grey", label="Random")
axes[0].set_title("ROC Curve")
axes[0].legend()

PrecisionRecallDisplay.from_estimator(best_model, X_test, y_test,
                                      ax=axes[1], color="#D9534F")
axes[1].set_title("Precision-Recall Curve")

plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_roc_pr_curves.png")
plt.show()


# Build a coefficient + odds-ratio table
coef_df = pd.DataFrame({
    "Feature"     : X_train.columns,
    "Coefficient" : best_model.coef_[0],
    "Abs_Coef"    : np.abs(best_model.coef_[0]),
    "Odds_Ratio"  : np.exp(best_model.coef_[0]),
}).sort_values("Abs_Coef", ascending=False).drop(columns="Abs_Coef")

print("Feature coefficients (sorted by magnitude):")
print(coef_df.round(4).to_string(index=False))

coef_df.to_csv(METRIC_DIR / "lr_coefficients.csv", index=False)

# Horizontal bar plot: green bars push towards heart disease, red bars push
away.
plot_df = coef_df.sort_values("Coefficient", ascending=True)
colors = ["#D9534F" if c < 0 else "#4C9F70" for c in
plot_df["Coefficient"]]

fig, ax = plt.subplots(figsize=(9, max(5, 0.35 * len(plot_df))))
ax.barh(plot_df["Feature"], plot_df["Coefficient"], color=colors)
ax.axvline(0, color="black", linewidth=0.8, linestyle="--")
ax.set_xlabel("Coefficient  (negative = lower risk,  positive = higher
risk)")
ax.set_title("Logistic Regression — Feature Coefficients")
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_feature_coefficients.png")
plt.show()


# Odds ratio plot — sometimes easier for a clinical audience.
plot_df2 = coef_df.sort_values("Odds_Ratio", ascending=True)
colors2 = ["#D9534F" if o < 1 else "#4C9F70" for o in
plot_df2["Odds_Ratio"]]

fig, ax = plt.subplots(figsize=(9, max(5, 0.35 * len(plot_df2))))
ax.barh(plot_df2["Feature"], plot_df2["Odds_Ratio"], color=colors2)
ax.axvline(1, color="black", linewidth=0.8, linestyle="--")
```

```
ax.set_xlabel("Odds Ratio  (< 1 = protective,  > 1 = risk factor)")
ax.set_title("Logistic Regression — Odds Ratios per Feature")
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_odds_ratios.png")
plt.show()


train_sizes, train_scores, val_scores = learning_curve(
    best_model, X_train, y_train,
    cv=cv,
    scoring="roc_auc",
    train_sizes=np.linspace(0.1, 1.0, 5),
    n_jobs=-1,
    random_state=RANDOM_STATE,
)

train_mean, train_std = train_scores.mean(axis=1), train_scores.std(axis=1)
val_mean,   val_std   = val_scores.mean(axis=1),   val_scores.std(axis=1)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(train_sizes, train_mean, "o-", color="#5B9BD5", label="Train ROC-
AUC")
ax.fill_between(train_sizes, train_mean - train_std, train_mean +
train_std,
                alpha=0.15, color="#5B9BD5")
ax.plot(train_sizes, val_mean, "o-", color="#D9534F", label="Validation
ROC-AUC")
ax.fill_between(train_sizes, val_mean - val_std, val_mean + val_std,
                alpha=0.15, color="#D9534F")
ax.set_xlabel("Training set size")
ax.set_ylabel("ROC-AUC")
ax.set_title("Learning Curve — Tuned Logistic Regression")
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig(PLOT_DIR / "lr_learning_curve.png")
plt.show()


# --- Save metrics ---
# Exact same JSON structure as the RF notebook for easy side-by-side
loading
# in the report's compare-and-contrast table.
all_metrics = {
    "baseline"       : baseline_metrics,
    "tuned"          : final_metrics,
    "best_params"    : grid_search.best_params_,
    "cv_mean_roc_auc": float(cv_scores.mean()),
    "cv_std_roc_auc" : float(cv_scores.std()),
}
with open(METRIC_DIR / "lr_metrics.json", "w") as f:
    json.dump(all_metrics, f, indent=4)

# Flat CSV (single-row) for quick viewing
pd.DataFrame([final_metrics]).to_csv(METRIC_DIR / "lr_metrics.csv",
index=False)

# --- Save predictions for the report's appendix / error analysis ---
pred_df = pd.DataFrame({
```

```
    "y_true": y_test.values,
    "y_pred": y_pred,
    "y_prob": y_prob,
})
pred_df.to_csv(METRIC_DIR / "lr_predictions.csv", index=False)

# --- Save model ---
joblib.dump(best_model, MODEL_DIR / "logistic_regression_model.pkl")

print("Saved:")
print(f"  metrics.json -> {METRIC_DIR / 'lr_metrics.json'}")
print(f"  metrics.csv  -> {METRIC_DIR / 'lr_metrics.csv'}")
print(f"  predictions  -> {METRIC_DIR / 'lr_predictions.csv'}")
print(f"  coefficients -> {METRIC_DIR / 'lr_coefficients.csv'}")
print(f"  model        -> {MODEL_DIR / 'logistic_regression_model.pkl'}")
print(f"  plots        -> {PLOT_DIR}")

logger.info("Logistic Regression notebook completed successfully.")
```

9.3 Notebook 3: Random Forest (random_forest.ipynb)

```
# Standard libs
import os
import json
import logging
from pathlib import Path

import pandas as pd
import numpy as np

# Visualisation
import matplotlib.pyplot as plt
import seaborn as sns

# ML libs
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold,
cross_val_score, learning_curve
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    PrecisionRecallDisplay,
    average_precision_score,
)
from sklearn.inspection import permutation_importance

# SMOTE (optional)
try:
    from imblearn.over_sampling import SMOTE
    from imblearn.pipeline import Pipeline as ImbPipeline
```

```
    HAS_SMOTE = True
except ImportError:
    print("imbalanced-learn not installed — SMOTE section will be
skipped.")
    print("Install with: pip install imbalanced-learn")
    HAS_SMOTE = False

# Save model
import joblib

# Reproducibility
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Plot defaults
sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)


BASE_DIR = Path(os.getcwd()).parent

DATA_DIR = BASE_DIR / "output" / "csv"
MODEL_DIR = BASE_DIR / "output" / "model"
METRIC_DIR = BASE_DIR / "output" / "metrics"
PLOT_DIR    = BASE_DIR / "output" / "plots"
LOG_DIR     = BASE_DIR / "output" / "logs"

MODEL_DIR.mkdir(parents=True, exist_ok=True)
METRIC_DIR.mkdir(parents=True, exist_ok=True)

print("DATA_DIR:", DATA_DIR)
print("MODEL_DIR:", MODEL_DIR)
print("PLOT_DIR :", PLOT_DIR)


# Append to the same log file the preprocessing notebook writes to,
log_file = LOG_DIR / "app.log"

for handler in logging.root.handlers[:]:
    logging.root.removeHandler(handler)

logging.basicConfig(
    filename=log_file,
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - [RandomForest] - %(message)s",
)
logger = logging.getLogger(__name__)
logger.info("Random Forest notebook started.")


# Load preprocessed data
X_train = pd.read_csv(DATA_DIR / "X_train.csv")
X_test = pd.read_csv(DATA_DIR / "X_test.csv")
y_train = pd.read_csv(DATA_DIR / "y_train.csv").squeeze()
y_test = pd.read_csv(DATA_DIR / "y_test.csv").squeeze()
```

```
print("Train Shape:", X_train.shape)
print("Test Shape:", X_test.shape)
logger.info(f"Loaded train={X_train.shape}, test={X_test.shape}")


assert not X_train.isnull().any().any(), "Missing values in X_train"
assert not X_test.isnull().any().any(), "Missing values in X_test"

assert len(X_train) == len(y_train), "Mismatch in training data"
assert len(X_test) == len(y_test), "Mismatch in test data"

assert set(y_train.unique()) <= {0, 1}, "Target should be binary 0/1"

print("Data validation passed")
logger.info("Data validation passed.")


# Show the class balance numerically and visually.
class_counts = y_train.value_counts().sort_index()
class_pct = (class_counts / len(y_train) * 100).round(2)

print("Class counts (train):")
print(pd.DataFrame({"count": class_counts, "percent": class_pct}))

# Visualise
fig, ax = plt.subplots(figsize=(6, 4))
sns.barplot(x=class_counts.index, y=class_counts.values,
            hue=class_counts.index, palette=["#4C9F70", "#D9534F"],
legend=False, ax=ax)
ax.set_xticks([0, 1])
ax.set_xticklabels(["No Heart Disease (0)", "Heart Disease (1)"])
ax.set_ylabel("Number of patients")
ax.set_title("Training set — class distribution")
for i, v in enumerate(class_counts.values):
    ax.text(i, v + max(class_counts.values) * 0.01,
f"{class_pct.iloc[i]}%",
            ha="center", fontweight="bold")
plt.tight_layout()
plt.savefig(PLOT_DIR / "class_distribution.png", dpi=120,
bbox_inches="tight")
plt.show()

# Compute imbalance ratio
imbalance_ratio = class_counts.max() / class_counts.min()
print(f"\nImbalance ratio (majority : minority) = {imbalance_ratio:.2f} :
1")
logger.info(f"Class imbalance ratio: {imbalance_ratio:.2f}:1")


# default RF with 100 trees, no class_weight (since imbalance ratio < 3:1)
baseline_model = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

baseline_model.fit(X_train, y_train)
```

```
baseline_pred = baseline_model.predict(X_test)
baseline_prob = baseline_model.predict_proba(X_test)[:, 1]

baseline_metrics = {
    "accuracy" : accuracy_score(y_test, baseline_pred),
    "precision": precision_score(y_test, baseline_pred, zero_division=0),
    "recall"   : recall_score(y_test, baseline_pred, zero_division=0),
    "f1"       : f1_score(y_test, baseline_pred, zero_division=0),
    "roc_auc"  : roc_auc_score(y_test, baseline_prob),
}
print("Baseline metrics:")
for k, v in baseline_metrics.items():
    print(f"  {k:9s}: {v:.4f}")

logger.info(f"Baseline metrics: {baseline_metrics}")


param_grid = {
    "n_estimators"     : [200, 400],          # more trees = more stable,
slower
    "max_depth"        : [None, 10, 20],      # None = grow until pure
    "min_samples_split": [2, 5],              # higher = more
regularisation
    "min_samples_leaf" : [1, 2],              # higher = smoother trees
    "max_features"     : ["sqrt", "log2"],    # how many features per split
    "class_weight"     : [None, "balanced"],  # handle class imbalance
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1),
    param_grid=param_grid,
    cv=cv,
    scoring="roc_auc",     # robust to class imbalance
    refit=True,            # refit best model on full training set
    verbose=1,
    n_jobs=-1,
    return_train_score=True,
)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_

print("Best CV ROC-AUC:", round(grid_search.best_score_, 4))
print("Best params    :", grid_search.best_params_)
logger.info(f"Best params: {grid_search.best_params_}")
logger.info(f"Best CV ROC-AUC: {grid_search.best_score_:.4f}")


cv_scores = cross_val_score(best_model, X_train, y_train, cv=cv,
                            scoring="roc_auc", n_jobs=-1)

print("Per-fold ROC-AUC :", np.round(cv_scores, 4))
print(f"Mean ROC-AUC     : {cv_scores.mean():.4f}")
print(f"Std  ROC-AUC     : {cv_scores.std():.4f}")
```

```
# Visual: boxplot of fold scores
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(y=cv_scores, color="#4C9F70", ax=ax)
sns.stripplot(y=cv_scores, color="black", size=8, ax=ax)
ax.set_ylabel("ROC-AUC per fold")
ax.set_title("5-fold CV ROC-AUC — Best Random Forest")
plt.tight_layout()
plt.savefig(PLOT_DIR / "cv_scores_boxplot.png", dpi=120,
bbox_inches="tight")
plt.show()


# Predictions on the test set
y_pred = best_model.predict(X_test)
y_prob = best_model.predict_proba(X_test)[:, 1]

# Headline metrics
final_metrics = {
    "accuracy" : accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, zero_division=0),
    "recall"   : recall_score(y_test, y_pred, zero_division=0),
    "f1"       : f1_score(y_test, y_pred, zero_division=0),
    "roc_auc"  : roc_auc_score(y_test, y_prob),
}

print("Tuned Random Forest — test metrics")
print("-" * 40)
for k, v in final_metrics.items():
    print(f"  {k:9s}: {v:.4f}")

print("\nClassification Report")
print("-" * 40)
print(classification_report(y_test, y_pred,
                            target_names=["No Heart Disease", "Heart
Disease"], zero_division=0))

logger.info(f"Final test metrics: {final_metrics}")


# Side-by-side comparison so the report can quantify the gain from tuning.
comparison_df = pd.DataFrame({
    "Baseline RF": baseline_metrics,
    "Tuned RF"   : final_metrics,
}).round(4)
comparison_df["Delta"] = (comparison_df["Tuned RF"] -
comparison_df["Baseline RF"]).round(4)
print(comparison_df)

fig, ax = plt.subplots(figsize=(8, 5))
comparison_df[["Baseline RF", "Tuned RF"]].plot(kind="bar", ax=ax,
                                                color=["#999999",
"#4C9F70"])
ax.set_ylabel("Score")
ax.set_title("Baseline vs Tuned Random Forest")
ax.set_ylim(0, 1)
ax.legend(loc="lower right")
plt.xticks(rotation=0)
```

```
plt.tight_layout()
plt.savefig(PLOT_DIR / "baseline_vs_tuned.png", dpi=120,
bbox_inches="tight")
plt.show()


cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No HD", "HD"],
            yticklabels=["No HD", "HD"], cbar=False, ax=ax)
ax.set_xlabel("Predicted label")
ax.set_ylabel("True label")
ax.set_title("Confusion Matrix — Tuned Random Forest")
plt.tight_layout()
plt.savefig(PLOT_DIR / "confusion_matrix.png", dpi=120,
bbox_inches="tight")
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True Negatives : {tn}")
print(f"False Positives: {fp}  (healthy patients incorrectly flagged)")
print(f"False Negatives: {fn}  (sick patients missed by model)")
print(f"True Positives : {tp}")


# ROC curve — true positive rate vs false positive rate at every threshold.
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

RocCurveDisplay.from_estimator(best_model, X_test, y_test,
                               ax=axes[0], curve_kwargs={"color":
"#4C9F70"})
axes[0].plot([0, 1], [0, 1], "--", color="grey", label="Random")
axes[0].set_title("ROC Curve")
axes[0].legend()

# Precision-Recall is more informative than ROC when classes are
imbalanced.
PrecisionRecallDisplay.from_estimator(best_model, X_test, y_test,
                                      ax=axes[1], color="#D9534F")
axes[1].set_title("Precision-Recall Curve")

plt.tight_layout()
plt.savefig(PLOT_DIR / "roc_pr_curves.png", dpi=120, bbox_inches="tight")
plt.show()


# Built-in Gini importance
feat_imp = pd.DataFrame({
    "feature"   : X_train.columns,
    "importance": best_model.feature_importances_,
}).sort_values("importance", ascending=False)

print("Top 10 features (Gini importance):")
print(feat_imp.head(10).to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 6))
sns.barplot(data=feat_imp.head(15), y="feature", x="importance",
            hue="feature", palette="viridis", legend=False, ax=ax)
```

```
ax.set_title("Random Forest — Gini Feature Importance (Top 15)")
plt.tight_layout()
plt.savefig(PLOT_DIR / "feature_importance_gini.png", dpi=120,
bbox_inches="tight")
plt.show()

feat_imp.to_csv(METRIC_DIR / "feature_importance.csv", index=False)


# Permutation importance (more reliable)
perm = permutation_importance(best_model, X_test, y_test,
                                n_repeats=10,
                                random_state=RANDOM_STATE,
                                n_jobs=-1,
                                scoring="roc_auc")

perm_df = pd.DataFrame({
    "feature": X_test.columns,
    "perm_importance_mean": perm.importances_mean,
    "perm_importance_std" : perm.importances_std,
}).sort_values("perm_importance_mean", ascending=False)

print("Top 10 features (Permutation importance):")
print(perm_df.head(10).to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 6))
sns.barplot(data=perm_df.head(15), y="feature", x="perm_importance_mean",
            hue="feature", palette="magma", legend=False, ax=ax)
ax.set_title("Random Forest — Permutation Importance (Top 15)")
ax.set_xlabel("Mean drop in ROC-AUC when feature is shuffled")
plt.tight_layout()
plt.savefig(PLOT_DIR / "feature_importance_permutation.png", dpi=120,
bbox_inches="tight")
plt.show()

perm_df.to_csv(METRIC_DIR / "feature_importance_permutation.csv",
index=False)


train_sizes, train_scores, val_scores = learning_curve(
    best_model, X_train, y_train,
    cv=cv,
    scoring="roc_auc",
    train_sizes=np.linspace(0.1, 1.0, 5),
    n_jobs=-1,
    random_state=RANDOM_STATE,
)

train_mean, train_std = train_scores.mean(axis=1), train_scores.std(axis=1)
val_mean,   val_std   = val_scores.mean(axis=1),   val_scores.std(axis=1)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(train_sizes, train_mean, "o-", color="#4C9F70", label="Train ROC-
AUC")
ax.fill_between(train_sizes, train_mean - train_std, train_mean +
train_std,
                alpha=0.15, color="#4C9F70")
```

```
ax.plot(train_sizes, val_mean, "o-", color="#D9534F", label="Validation
ROC-AUC")
ax.fill_between(train_sizes, val_mean - val_std, val_mean + val_std,
                alpha=0.15, color="#D9534F")
ax.set_xlabel("Training set size")
ax.set_ylabel("ROC-AUC")
ax.set_title("Learning Curve — Tuned Random Forest")
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig(PLOT_DIR / "learning_curve.png", dpi=120, bbox_inches="tight")
plt.show()


# Save metrics
all_metrics = {
    "baseline": baseline_metrics,
    "tuned"   : final_metrics,
    "best_params": grid_search.best_params_,
    "cv_mean_roc_auc": float(cv_scores.mean()),
    "cv_std_roc_auc" : float(cv_scores.std()),
}
with open(METRIC_DIR / "rf_metrics.json", "w") as f:
    json.dump(all_metrics, f, indent=4)

pd.DataFrame([final_metrics]).to_csv(METRIC_DIR / "metrics.csv",
index=False)

# Save predictions for the report's appendix / error analysis
pred_df = pd.DataFrame({
    "y_true": y_test.values,
    "y_pred": y_pred,
    "y_prob": y_prob,
})
pred_df.to_csv(METRIC_DIR / "predictions.csv", index=False)

# Save model
joblib.dump(best_model, MODEL_DIR / "random_forest_model.pkl")

print("Saved:")
print(f"  metrics.json -> {METRIC_DIR / 'rf_metrics.json'}")
print(f"  metrics.csv  -> {METRIC_DIR / 'metrics.csv'}")
print(f"  predictions  -> {METRIC_DIR / 'predictions.csv'}")
print(f"  model        -> {MODEL_DIR / 'random_forest_model.pkl'}")
print(f"  plots        -> {PLOT_DIR}")

logger.info("Random Forest notebook completed successfully.")
```